

Delegate - C++11

Generated by Doxygen 1.16.1

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Namespace Documentation	9
5.1 bricxx Namespace Reference	9
6 Class Documentation	11
6.1 bricxx::BadDelegateCall Class Reference	11
6.1.1 Detailed Description	11
6.1.2 Member Function Documentation	11
6.1.2.1 what()	11
6.2 bricxx::Delegate< TSignature > Class Template Reference	12
6.2.1 Detailed Description	12
6.3 bricxx::Delegate< TRetVal(TParams...) const > Class Template Reference	13
6.3.1 Detailed Description	15
6.3.2 Constructor & Destructor Documentation	15
6.3.2.1 Delegate() [1/2]	15
6.3.2.2 Delegate() [2/2]	15
6.3.2.3 ~Delegate()	15
6.3.3 Member Function Documentation	16
6.3.3.1 operator=()	16
6.3.3.2 operator==()	16
6.3.3.3 operator!=(())	16
6.3.3.4 Bind() [1/3]	16
6.3.3.5 Bind() [2/3]	16
6.3.3.6 Bind() [3/3]	16
6.3.3.7 IsBindedTo() [1/3]	17
6.3.3.8 IsBindedTo() [2/3]	17
6.3.3.9 IsBindedTo() [3/3]	17
6.3.3.10 operator()()	17
6.3.3.11 CreateAndBind() [1/3]	17
6.3.3.12 CreateAndBind() [2/3]	18
6.3.3.13 CreateAndBind() [3/3]	18
6.4 bricxx::Delegate< TRetVal(TParams...) > Class Template Reference	18
6.4.1 Detailed Description	19

6.4.2 Constructor & Destructor Documentation	20
6.4.2.1 Delegate() [1/2]	20
6.4.2.2 Delegate() [2/2]	20
6.4.2.3 ~Delegate()	20
6.4.3 Member Function Documentation	20
6.4.3.1 operator=()	20
6.4.3.2 operator==()	20
6.4.3.3 operator!=(())	20
6.4.3.4 Bind() [1/3]	21
6.4.3.5 Bind() [2/3]	21
6.4.3.6 Bind() [3/3]	21
6.4.3.7 IsBindedTo() [1/3]	21
6.4.3.8 IsBindedTo() [2/3]	21
6.4.3.9 IsBindedTo() [3/3]	21
6.4.3.10 operator()()	22
6.4.3.11 CreateAndBind() [1/3]	22
6.4.3.12 CreateAndBind() [2/3]	22
6.4.3.13 CreateAndBind() [3/3]	22
6.5 bricxx::DelegateBase< TRetVal, TParams > Class Template Reference	23
6.5.1 Detailed Description	24
6.5.2 Member Typedef Documentation	24
6.5.2.1 ErasedObjectType	24
6.5.2.2 StubFnPtr	24
6.5.3 Constructor & Destructor Documentation	24
6.5.3.1 DelegateBase() [1/2]	24
6.5.3.2 DelegateBase() [2/2]	24
6.5.3.3 ~DelegateBase()	24
6.5.4 Member Function Documentation	25
6.5.4.1 IsBinded()	25
6.5.4.2 operator bool()	25
6.5.4.3 Unbind()	25
6.5.4.4 Reset()	25
6.5.4.5 operator=()	25
6.5.4.6 operator==()	25
6.5.4.7 operator!=(())	25
6.5.4.8 NullStub()	26
6.5.5 Member Data Documentation	26
6.5.5.1 pStub_	26
6.5.5.2 pTypeErasedObject_	26
7 File Documentation	27
7.1 delegate.hpp File Reference	27

7.1.1 Macro Definition Documentation	27
7.1.1.1 BRICXX_DELEGATE_API	27
7.2 delegate.hpp	28
Index	37

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

bricxx	9
----------------------------------	---

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

<code>bricxx::Delegate< TSignature ></code>	12
<code>bricxx::DelegateBase< TRetVal, TParams ></code>	23
<code>bricxx::DelegateBase< TRetVal, TParams... ></code>	23
<code>bricxx::Delegate< TRetVal(TParams...) const ></code>	13
<code>bricxx::Delegate< TRetVal(TParams...) ></code>	18
<code>std::exception</code>	
<code>bricxx::BadDelegateCall</code>	11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

bricxx::BadDelegateCall	Exception thrown by <code>Delegate::operator()()</code> if the delegate is not binded	11
bricxx::Delegate< TSignature >	A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined	12
bricxx::Delegate< TRetVal(TParams...) const >	Delegate partial specialization for const	13
bricxx::Delegate< TRetVal(TParams...) >	Delegate partial specialization for non-const	18
bricxx::DelegateBase< TRetVal, TParams >	Base class for Delegate partial specializations	23

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

delegate.hpp	27
------------------------------	----

Chapter 5

Namespace Documentation

5.1 bricxx Namespace Reference

Classes

- class [Delegate](#)
A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined.
- class [BadDelegateCall](#)
Exception thrown by `Delegate::operator()()` if the delegate is not binded.
- class [DelegateBase](#)
Base class for `Delegate` partial specializations.
- class [Delegate< TRetVal\(TParams...\)>](#)
`Delegate` partial specialization for non-const.
- class [Delegate< TRetVal\(TParams...\) const >](#)
`Delegate` partial specialization for const.

Chapter 6

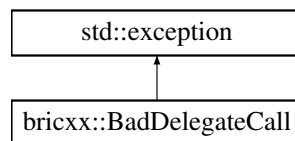
Class Documentation

6.1 bricxx::BadDelegateCall Class Reference

Exception thrown by `Delegate::operator()()` if the delegate is not binded.

```
#include <delegate.hpp>
```

Inheritance diagram for `bricxx::BadDelegateCall`:



Public Member Functions

- `const char * what() const` noexcept override

6.1.1 Detailed Description

Exception thrown by `Delegate::operator()()` if the delegate is not binded.

Definition at line [161](#) of file [delegate.hpp](#).

6.1.2 Member Function Documentation

6.1.2.1 what()

```
const char * bricxx::BadDelegateCall::what() const [override], [noexcept]
```

The documentation for this class was generated from the following file:

- [delegate.hpp](#)

6.2 bricxx::Delegate< TSignature > Class Template Reference

A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined.

6.2.1 Detailed Description

```
template<typename TSignature>
class bricxx::Delegate< TSignature >
```

A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined.

[Delegate](#):

- is copyable, small (two pointers) and fast (no dynamic memory allocation, no virtual functions, all functions are inline (except stubs because their address is needed), delegate call involves only two extra calls (operator>() and stub function)).
- supports (member) functions and functors/lambda.
- is a single header library so it is easy to integrate in any project.

Attention

Because [Delegate](#) is non-owning special care is needed to ensure that the binded target outlive it.

Remarks

- Two delegates are considered equal if they are not binded or binded to the same target.
- Equality comparison and `Delegate::IsBindedTo()` allows delegates to be easily used as call-backs that can be registered/unregistered (`::std::function_ref` is not equality comparable).
- The solutions presented in **references** directory are not const-correct (they have the same problem as `::std::function`).

See also

- [Rich Hickey: Callbacks in C++ Using Template Functors](#)
- Bert Peers: [Callbacks in C++](#)
- Don Clugston, CodeProject: [Member Function Pointers and the Fastest Possible C++ Delegates](#)
- Sergey Ryazanov, CodeProject: [The Impossibly Fast C++ Delegates](#)
- Sergey Alexandrovich Kryukov, CodeProject: [The Impossibly Fast C++ Delegates, Fixed](#)
- Aleksei Trunov, CodeProject: [Yet Another Generalized Functors Implementation in C++](#)
- JaeWook Choi, CodeProject: [Fast C++ Delegate](#)
- [Matthew Rodusek: Creating a Fast and Efficient Delegate Type \(Part 1\)](#)
- [Matthew Rodusek: Creating a Fast and Efficient Delegate Type \(Part 2\)](#)
- [Matthew Rodusek: Creating a Fast and Efficient Delegate Type \(Part 3\)](#)
- [Matthew Rodusek: Unbelievably Fast Delegate](#)

- [user1095108](#): Impossibly fast delegate in C++11
- [user1095108](#): [delegate.hpp](#)
- **references** directory in this project that contains the above.

Minimal introduction/examples (please see tests for more details):

```
// (static member) function
Delegate<int (int)> functionDelegate;
functionDelegate.Bind<&::islower>();
if (!functionDelegate.IsBindedTo<&::islower>()) throw runtime_error("function test failed");
if (!functionDelegate('a')) throw runtime_error("function test failed");

// lambda/functor
unsigned int numCalls = 0U;
auto lambda = [&numCalls](int x, int y) mutable -> bool {
    ++numCalls;
    return x > y;
};
Delegate<bool (int, int)> lambdaDelegate;
lambdaDelegate.Bind(lambda);
if (!lambdaDelegate.IsBindedTo(lambda)) throw runtime_error("lambda test failed");
if (!lambdaDelegate(2, 1)) throw runtime_error("lambda test failed");
if (numCalls != 1U) throw runtime_error("lambda test failed");

// non-static member function
string hello = "Hello, World!";
Delegate<void ()> methodDelegate;
methodDelegate.Bind<string, &string::clear>(hello);
methodDelegate();
if (!hello.empty()) throw runtime_error("method test failed");

// comparison
Delegate<int (int)> functionDelegate1;
functionDelegate1.Bind<&::isupper>();
if (functionDelegate == functionDelegate1) throw runtime_error("comparison test failed");
functionDelegate1.Bind<&::islower>();
if (functionDelegate != functionDelegate1) throw runtime_error("comparison test failed");

// const-correctness
Delegate<void () const> constDelegate;
//constDelegate.Bind<string, &string::clear>(hello); // OK, does not compile
(void)constDelegate;

// unbind
functionDelegate.Unbind();
if (functionDelegate.IsBinded()) throw runtime_error("unbind test failed");

// BadDelegateCall
bool caught = false;
try {
    functionDelegate('a');
}
catch (const BadDelegateCall&) {
    caught = true;
}
if (!caught) throw runtime_error("BadDelegateCall test failed");
```

Definition at line 154 of file [delegate.hpp](#).

The documentation for this class was generated from the following file:

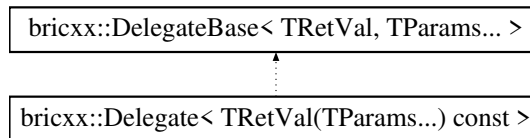
- [delegate.hpp](#)

6.3 bricxx::Delegate< TRetVal(TParams...) const > Class Template Reference

[Delegate](#) partial specialization for const.

```
#include <delegate.hpp>
```

Inheritance diagram for bricxx::Delegate< TRetVal(TParams...) const >:



Public Member Functions

- `~Delegate ()` noexcept=default
- `Delegate & operator= (const Delegate &other)` noexcept
- `template<typename... UParams>`
`TRetVal operator() (UParams &&... params)` const

Constructors.

- `constexpr Delegate ()` noexcept
Creates a delegate that is not binded.
- `constexpr Delegate (const Delegate &other)` noexcept

Equality operators.

- `constexpr bool operator== (const Delegate &other)` const noexcept
- `constexpr bool operator!= (const Delegate &other)` const noexcept

Binders.

Binds the delegate to a target.

- `template<TRetVal(*) (TParams...) TFunction>`
`void Bind ()` noexcept
Binds the delegate to a (static member) function.
- `template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::value >::type>`
`void Bind (const TFunction &functor)` noexcept
Binds the delegate to a functor / lambda (TFunction is auto-deduced).
- `template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>`
`void Bind (const TClass &object)` noexcept
Binds the delegate to an object + non-static member function.

Binder target testers (convenience methods).

Test if the delegate is binded to the given target.

- `template<TRetVal(*) (TParams...) TFunction>`
`bool IsBindedTo ()` const noexcept
Test if the delegate is binded to the given (static member) function.
- `template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::value >::type>`
`bool IsBindedTo (const TFunction &functor)` const noexcept
Test if the delegate is binded to the given functor / lambda (TFunction is auto-deduced).
- `template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>`
`bool IsBindedTo (const TClass &object)` const noexcept
Test if the delegate is binded to the given object + non-static member function.

Static Public Member Functions

Create and bind (convenience methods).

Creates a delegate and binds it to the given target.

- `template<TRetVal(*)(TParams...) TFunction>`
`static Delegate CreateAndBind () noexcept`
Creates a delegate and binds it to the given (static member) function.
- `template<class TFunc, typename = typename ::std::enable_if< ::std::is_class<TFunc>::value >::type>`
`static Delegate CreateAndBind (const TFunc &func) noexcept`
Creates a delegate and binds it to the given functor / lambda (TFunc is auto-deduced).
- `template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>`
`static Delegate CreateAndBind (const TClass &object) noexcept`
Creates a delegate and binds it to the given object + non-static member function.

6.3.1 Detailed Description

```
template<typename TRetVal, typename... TParams>
class bricxx::Delegate< TRetVal(TParams...) const >
```

[Delegate](#) partial specialization for const.

Definition at line 339 of file [delegate.hpp](#).

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Delegate() [1/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const >::Delegate () [constexpr], [noexcept]
```

Creates a delegate that is not binded.

6.3.2.2 Delegate() [2/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const >::Delegate (
    const Delegate< TRetVal(TParams...) const > & other) [constexpr], [noexcept]
```

6.3.2.3 ~Delegate()

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const >::~~Delegate () [default], [noexcept]
```

Remarks

Not virtual.

6.3.3 Member Function Documentation

6.3.3.1 operator=()

```
template<typename TRetVal, typename... TParams>
Delegate & bricxx::Delegate< TRetVal(TParams...) const >::operator= (
    const Delegate< TRetVal(TParams...) const > & other) [noexcept]
```

6.3.3.2 operator==(())

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...) const >::operator== (
    const Delegate< TRetVal(TParams...) const > & other) const [constexpr], [noexcept]
```

6.3.3.3 operator!=(())

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...) const >::operator!= (
    const Delegate< TRetVal(TParams...) const > & other) const [constexpr], [noexcept]
```

6.3.3.4 Bind() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
void bricxx::Delegate< TRetVal(TParams...) const >::Bind () [noexcept]
```

Binds the delegate to a (static member) function.

6.3.3.5 Bind() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFuncor, typename = typename ::std::enable_if< ::std::is_class<TFuncor>::value >::type>
void bricxx::Delegate< TRetVal(TParams...) const >::Bind (
    const TFuncor & functor) [noexcept]
```

Binds the delegate to a functor / lambda (TFuncor is auto-deduced).

6.3.3.6 Bind() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
void bricxx::Delegate< TRetVal(TParams...) const >::Bind (
    const TClass & object) [noexcept]
```

Binds the delegate to an object + non-static member function.

6.3.3.7 IsBindedTo() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
bool bricxx::Delegate< TRetVal(TParams...) const >::IsBindedTo () const [noexcept]
```

Test if the delegate is binded to the given (static member) function.

6.3.3.8 IsBindedTo() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::↵
value >::type>
bool bricxx::Delegate< TRetVal(TParams...) const >::IsBindedTo (
    const TFunction & function) const [noexcept]
```

Test if the delegate is binded to the given *function* / lambda (TFunction is auto-deduced).

6.3.3.9 IsBindedTo() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
bool bricxx::Delegate< TRetVal(TParams...) const >::IsBindedTo (
    const TClass & object) const [noexcept]
```

Test if the delegate is binded to the given *object* + non-static member function.

6.3.3.10 operator()()

```
template<typename TRetVal, typename... TParams>
template<typename... UParams>
TRetVal bricxx::Delegate< TRetVal(TParams...) const >::operator() (
    UParams &&... params) const
```

Invokes the stub function with the given parameters.

Remarks

Perfect forwarding and converting parameter pack to stub function (UParams is a deduced type, TParams is not).

Exceptions

<i>BadDelegateCall</i>	if the delegate is not binded.
--	--------------------------------

6.3.3.11 CreateAndBind() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
Delegate bricxx::Delegate< TRetVal(TParams...) const >::CreateAndBind () [static], [noexcept]
```

Creates a delegate and binds it to the given (static member) function.

6.3.3.12 CreateAndBind() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFunc, typename = typename ::std::enable_if< ::std::is_class<TFunc>>::type>
value >::type>
Delegate bricxx::Delegate< TRetVal(TParams...) const >::CreateAndBind (
    const TFunc & func) [static], [noexcept]
```

Creates a delegate and binds it to the given `func` / `lambda` (`TFunc` is auto-deduced).

6.3.3.13 CreateAndBind() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
Delegate bricxx::Delegate< TRetVal(TParams...) const >::CreateAndBind (
    const TClass & obj) [static], [noexcept]
```

Creates a delegate and binds it to the given `obj` + non-static member function.

The documentation for this class was generated from the following file:

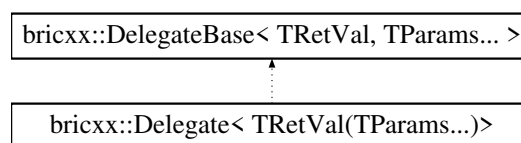
- [delegate.hpp](#)

6.4 bricxx::Delegate< TRetVal(TParams...)> Class Template Reference

`Delegate` partial specialization for non-const.

```
#include <delegate.hpp>
```

Inheritance diagram for `bricxx::Delegate< TRetVal(TParams...)>`:



Public Member Functions

- `~Delegate ()` noexcept=default
- `Delegate & operator= (const Delegate &other)` noexcept
- `template<typename... UParams>`
`TRetVal operator() (UParams &&... params)`

Constructors.

- `constexpr Delegate ()` noexcept
Creates a delegate that is not binded.
- `constexpr Delegate (const Delegate &other)` noexcept

Equality operators.

- constexpr bool `operator==` (const [Delegate](#) &other) const noexcept
- constexpr bool `operator!=` (const [Delegate](#) &other) const noexcept

Binders.

Binds the delegate to a target.

- template<TRetVal(*) (TParams...) TFunction>
void `Bind` () noexcept
Binds the delegate to a (static member) function.
- template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::value >::type>
void `Bind` (TFunction &function) noexcept
Binds the delegate to a functor / lambda (TFunction is auto-deduced).
- template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
void `Bind` (TClass &object) noexcept
Binds the delegate to an object + non-static member function.

Binder target testers (convenience methods).

Test if the delegate is binded to the given target.

- template<TRetVal(*) (TParams...) TFunction>
bool `IsBindedTo` () const noexcept
Test if the delegate is binded to the given (static member) function.
- template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::value >::type>
bool `IsBindedTo` (TFunction &function) const noexcept
Test if the delegate is binded to the given functor / lambda (TFunction is auto-deduced).
- template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
bool `IsBindedTo` (TClass &object) const noexcept
Test if the delegate is binded to the given object + non-static member function.

Static Public Member Functions**Create and bind (convenience methods).**

Creates a delegate and binds it to the given target.

- template<TRetVal(*) (TParams...) TFunction>
static [Delegate](#) `CreateAndBind` () noexcept
Creates a delegate and binds it to the given (static member) function.
- template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::value >::type>
static [Delegate](#) `CreateAndBind` (TFunction &function) noexcept
Creates a delegate and binds it to the given functor / lambda (TFunction is auto-deduced).
- template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
static [Delegate](#) `CreateAndBind` (TClass &object) noexcept
Creates a delegate and binds it to the given object + non-static member function.

6.4.1 Detailed Description

```
template<typename TRetVal, typename... TParams>
class bricxx::Delegate< TRetVal(TParams...)>
```

[Delegate](#) partial specialization for non-const.

Definition at line 216 of file [delegate.hpp](#).

6.4.2 Constructor & Destructor Documentation

6.4.2.1 Delegate() [1/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...)>::Delegate () [constexpr], [noexcept]
```

Creates a delegate that is not binded.

6.4.2.2 Delegate() [2/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...)>::Delegate (
    const Delegate< TRetVal(TParams...)> & other) [constexpr], [noexcept]
```

6.4.2.3 ~Delegate()

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...)>::~~Delegate () [default], [noexcept]
```

Remarks

Not virtual.

6.4.3 Member Function Documentation

6.4.3.1 operator=()

```
template<typename TRetVal, typename... TParams>
Delegate & bricxx::Delegate< TRetVal(TParams...)>::operator= (
    const Delegate< TRetVal(TParams...)> & other) [noexcept]
```

6.4.3.2 operator==()

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...)>::operator== (
    const Delegate< TRetVal(TParams...)> & other) const [constexpr], [noexcept]
```

6.4.3.3 operator!=(())

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...)>::operator!= (
    const Delegate< TRetVal(TParams...)> & other) const [constexpr], [noexcept]
```

6.4.3.4 Bind() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
void bricxx::Delegate< TRetVal(TParams...)>::Bind () [noexcept]
```

Binds the delegate to a (static member) function.

6.4.3.5 Bind() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::↵
value >::type>
void bricxx::Delegate< TRetVal(TParams...)>::Bind (
    TFunction & function) [noexcept]
```

Binds the delegate to a functor / lambda (TFunction is auto-deduced).

6.4.3.6 Bind() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
void bricxx::Delegate< TRetVal(TParams...)>::Bind (
    TClass & object) [noexcept]
```

Binds the delegate to an object + non-static member function.

6.4.3.7 IsBindedTo() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
bool bricxx::Delegate< TRetVal(TParams...)>::IsBindedTo () const [noexcept]
```

Test if the delegate is binded to the given (static member) function.

6.4.3.8 IsBindedTo() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFunction, typename = typename ::std::enable_if< ::std::is_class<TFunction>::↵
value >::type>
bool bricxx::Delegate< TRetVal(TParams...)>::IsBindedTo (
    TFunction & function) const [noexcept]
```

Test if the delegate is binded to the given functor / lambda (TFunction is auto-deduced).

6.4.3.9 IsBindedTo() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
bool bricxx::Delegate< TRetVal(TParams...)>::IsBindedTo (
    TClass & object) const [noexcept]
```

Test if the delegate is binded to the given object + non-static member function.

6.4.3.10 operator()

```
template<typename TRetVal, typename... TParams>
template<typename... UParams>
TRetVal bricxx::Delegate< TRetVal(TParams...)>::operator() (
    UParams &&... params)
```

Invokes the stub function with the given parameters.

Remarks

Perfect forwarding and converting parameter pack to stub function (UParams is a deduced type, TParams is not).

Exceptions

BadDelegateCall	if the delegate is not binded.
---------------------------------	--------------------------------

6.4.3.11 CreateAndBind() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*) (TParams...) TFunction>
Delegate bricxx::Delegate< TRetVal(TParams...)>::CreateAndBind () [static], [noexcept]
```

Creates a delegate and binds it to the given (static member) function.

6.4.3.12 CreateAndBind() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFuncor, typename = typename ::std::enable_if< ::std::is_class<TFuncor>>::↔
value >::type>
Delegate bricxx::Delegate< TRetVal(TParams...)>::CreateAndBind (
    TFuncor & functor) [static], [noexcept]
```

Creates a delegate and binds it to the given functor / lambda (TFuncor is auto-deduced).

6.4.3.13 CreateAndBind() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
Delegate bricxx::Delegate< TRetVal(TParams...)>::CreateAndBind (
    TClass & object) [static], [noexcept]
```

Creates a delegate and binds it to the given object + non-static member function.

The documentation for this class was generated from the following file:

- [delegate.hpp](#)

6.5 bricxx::DelegateBase< TRetVal, TParams > Class Template Reference

Base class for [Delegate](#) partial specializations.

```
#include <delegate.hpp>
```

Public Member Functions

- constexpr bool [IsBinded](#) () const noexcept
- constexpr [operator bool](#) () const noexcept
Same as [IsBinded\(\)](#).
- void [Unbind](#) () noexcept
- void [Reset](#) () noexcept
Same as [Unbind\(\)](#).

Protected Types

- using [ErasedObjectType](#) = void
- using [StubFnPtr](#) = TRetVal (*)([ErasedObjectType](#)* pTypeErasedObject, TParams... params)

Protected Member Functions

- [~DelegateBase](#) () noexcept=default
- [DelegateBase](#) & [operator=](#) (const [DelegateBase](#) &other) noexcept

Constructors.

- constexpr [DelegateBase](#) () noexcept
Creates a delegate that is not binded.
- constexpr [DelegateBase](#) (const [DelegateBase](#) &other) noexcept

Equality operators.

- constexpr bool [operator==](#) (const [DelegateBase](#) &other) const noexcept
- constexpr bool [operator!=](#) (const [DelegateBase](#) &other) const noexcept

Static Protected Member Functions

- static TRetVal [NullStub](#) ([ErasedObjectType](#) *pTypeErasedObject, TParams... params)

Protected Attributes

- [StubFnPtr](#) [pStub_](#) = &[NullStub](#)
- [ErasedObjectType](#) * [pTypeErasedObject_](#) = nullptr

6.5.1 Detailed Description

```
template<typename TRetVal, typename... TParams>
class bricxx::DelegateBase< TRetVal, TParams >
```

Base class for [Delegate](#) partial specializations.

Definition at line 172 of file [delegate.hpp](#).

6.5.2 Member Typedef Documentation

6.5.2.1 ErasedObjectType

```
template<typename TRetVal, typename... TParams>
using bricxx::DelegateBase< TRetVal, TParams >::ErasedObjectType = void [protected]
```

Definition at line 180 of file [delegate.hpp](#).

6.5.2.2 StubFnPtr

```
template<typename TRetVal, typename... TParams>
using bricxx::DelegateBase< TRetVal, TParams >::StubFnPtr = TRetVal (*) (ErasedObjectType* p↔
TypeErasedObject, TParams... params) [protected]
```

Definition at line 181 of file [delegate.hpp](#).

6.5.3 Constructor & Destructor Documentation

6.5.3.1 DelegateBase() [1/2]

```
template<typename TRetVal, typename... TParams>
bricxx::DelegateBase< TRetVal, TParams >::DelegateBase () [constexpr], [protected], [noexcept]
```

Creates a delegate that is not binded.

6.5.3.2 DelegateBase() [2/2]

```
template<typename TRetVal, typename... TParams>
bricxx::DelegateBase< TRetVal, TParams >::DelegateBase (
    const DelegateBase< TRetVal, TParams > & other) [constexpr], [protected], [noexcept]
```

6.5.3.3 ~DelegateBase()

```
template<typename TRetVal, typename... TParams>
bricxx::DelegateBase< TRetVal, TParams >::~~DelegateBase () [protected], [default], [noexcept]
```

Remarks

Not virtual.

6.5.4 Member Function Documentation

6.5.4.1 IsBinded()

```
template<typename TRetVal, typename... TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::IsBinded () const [constexpr], [noexcept]
```

6.5.4.2 operator bool()

```
template<typename TRetVal, typename... TParams>
bricxx::DelegateBase< TRetVal, TParams >::operator bool () const [explicit], [constexpr],
[noexcept]
```

Same as `IsBinded()`.

6.5.4.3 Unbind()

```
template<typename TRetVal, typename... TParams>
void bricxx::DelegateBase< TRetVal, TParams >::Unbind () [noexcept]
```

6.5.4.4 Reset()

```
template<typename TRetVal, typename... TParams>
void bricxx::DelegateBase< TRetVal, TParams >::Reset () [noexcept]
```

Same as `Unbind()`.

6.5.4.5 operator=()

```
template<typename TRetVal, typename... TParams>
DelegateBase & bricxx::DelegateBase< TRetVal, TParams >::operator= (
    const DelegateBase< TRetVal, TParams > & other) [protected], [noexcept]
```

6.5.4.6 operator==()

```
template<typename TRetVal, typename... TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::operator== (
    const DelegateBase< TRetVal, TParams > & other) const [constexpr], [protected],
[noexcept]
```

6.5.4.7 operator!="()

```
template<typename TRetVal, typename... TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::operator!= (
    const DelegateBase< TRetVal, TParams > & other) const [constexpr], [protected],
[noexcept]
```

6.5.4.8 NullStub()

```
template<typename TRetVal, typename... TParams>
TRetVal bricxx::DelegateBase< TRetVal, TParams >::NullStub (
    ErasedObjectType * pTypeErasedObject,
    TParams... params) [static], [protected]
```

Exceptions

<i>BadDelegateCall</i>	This stub is used when the delegate is not binded to a "real" target.
--	---

6.5.5 Member Data Documentation

6.5.5.1 pStub_

```
template<typename TRetVal, typename... TParams>
StubFnPtr bricxx::DelegateBase< TRetVal, TParams >::pStub_ = &NullStub [protected]
```

Definition at line 206 of file [delegate.hpp](#).

6.5.5.2 pTypeErasedObject_

```
template<typename TRetVal, typename... TParams>
ErasedObjectType* bricxx::DelegateBase< TRetVal, TParams >::pTypeErasedObject_ = nullptr [protected]
```

Definition at line 207 of file [delegate.hpp](#).

The documentation for this class was generated from the following file:

- [delegate.hpp](#)

Chapter 7

File Documentation

7.1 `delegate.hpp` File Reference

```
#include <exception>
#include <type_traits>
#include <utility>
#include <memory>
```

Classes

- class `bricxx::BadDelegateCall`
Exception thrown by `Delegate::operator()()` if the delegate is not binded.
- class `bricxx::DelegateBase< TRetVal, TParams >`
Base class for `Delegate` partial specializations.
- class `bricxx::Delegate< TRetVal(TParams...) >`
`Delegate` partial specialization for non-const.
- class `bricxx::Delegate< TRetVal(TParams...) const >`
`Delegate` partial specialization for const.

Namespaces

- namespace `bricxx`

Macros

- `#define BRICXX_DELEGATE_API`

7.1.1 Macro Definition Documentation

7.1.1.1 `BRICXX_DELEGATE_API`

```
#define BRICXX_DELEGATE_API
```

Definition at line 58 of file `delegate.hpp`.

7.2 delegate.hpp

[Go to the documentation of this file.](#)

```

00001 // MIT License: https://spdx.org/licenses/MIT.html
00002 //
00003 // Copyright (c) 2026 Cristian ANITA <cristian_anita@yahoo.com>
00004 //
00005 // Permission is hereby granted, free of charge, to any person obtaining a copy
00006 // of this software and associated documentation files (the "Software"), to deal
00007 // in the Software without restriction, including without limitation the rights
00008 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00009 // copies of the Software, and to permit persons to whom the Software is
00010 // furnished to do so, subject to the following conditions:
00011 //
00012 // The above copyright notice and this permission notice shall be included in all
00013 // copies or substantial portions of the Software.
00014 //
00015 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
00016 // EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
00017 // MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
00018 // IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,
00019 // DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR
00020 // OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE
00021 // OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023
00024 #ifndef BRICXX_DELEGATE_HPP_INCLUDED_
00025 #define BRICXX_DELEGATE_HPP_INCLUDED_
00026
00027
00028 // /Zc:__cplusplus; please see:
00029 // - MSVC now correctly reports __cplusplus:
00030 // https://devblogs.microsoft.com/cppblog/msvc-now-correctly-reports-__cplusplus
00031 // - /Zc:__cplusplus (Enable updated __cplusplus macro):
00032 // https://learn.microsoft.com/en-us/cpp/build/reference/zc-cplusplus
00033 #ifdef _MSC_VER
00034 #   if (_MSC_VER < 1914)
00035 #       error "msvc >= 2017 v15.7 required!!! (for /Zc:__cplusplus)"
00036 #   endif
00037 #endif
00038
00039 #if (__cplusplus < 201103L)
00040 #   error "C++ >= 11 required!!!"
00041 #endif
00042
00043 #ifdef _MSC_VER
00044 #pragma warning(push)
00045 // C4646: function declared with __declspec(noreturn) has non-void return type.
00046 // DelegateBase::NullStub() has (possibly non-void) return type (it must conform to stub signature)
00047 // but it actually always throws BadDelegateCall.
00048 #pragma warning(disable: 4646)
00049 #endif
00050
00051 #include <exception>
00052 #include <type_traits>
00053 #include <utility>
00054 #include <memory>
00055
00056
00057 // TODO: If Delegate is used in a shared library define BRICXX_DELEGATE_API to that library
00058 // export/import macro.
00059 #define BRICXX_DELEGATE_API
00060
00061 namespace bricxx {
00062
00063
00064 //*****
00153 template <typename TSignature>
00154 class Delegate;
00155
00156
00157
00158
00159 //*****
00161 class BRICXX_DELEGATE_API BadDelegateCall final: public ::std::exception {
00162 public:
00163     const char* what() const noexcept override;
00164 };
00165
00166

```

```

00167
00168
00169
00171 //*****
00172 template <typename TRetVal, typename... TParams>
00173 class DelegateBase {
00174 public:
00175     constexpr bool IsBinded() const noexcept;
00176     explicit constexpr operator bool() const noexcept;
00177     void Unbind() noexcept;
00178     void Reset() noexcept;
00179 protected:
00180     using ErasedObjectType = void;
00181     using StubFnPtr = TRetVal (*)(ErasedObjectType* pTypeErasedObject, TParams... params);
00182
00183     constexpr DelegateBase() noexcept;
00184     constexpr DelegateBase(const DelegateBase& other) noexcept;
00185
00186     ~DelegateBase() noexcept = default;
00187     DelegateBase& operator=(const DelegateBase& other) noexcept;
00188
00189     constexpr bool operator==(const DelegateBase& other) const noexcept;
00190     constexpr bool operator!=(const DelegateBase& other) const noexcept;
00191
00192     [[noreturn]] static TRetVal NullStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00193
00194     StubFnPtr pStub_ = &NullStub;
00195     ErasedObjectType* pTypeErasedObject_ = nullptr;
00196 };
00197
00198
00199
00200 //*****
00201 template <typename TRetVal, typename... TParams>
00202 class Delegate<TRetVal (TParams...)> final: private DelegateBase<TRetVal, TParams...> {
00203 public:
00204     using DelegateBase<TRetVal, TParams...>::IsBinded;
00205     using DelegateBase<TRetVal, TParams...>::operator bool;
00206
00207     using DelegateBase<TRetVal, TParams...>::Unbind;
00208     using DelegateBase<TRetVal, TParams...>::Reset;
00209
00210     constexpr Delegate() noexcept;
00211     constexpr Delegate(const Delegate& other) noexcept;
00212
00213     ~Delegate() noexcept = default;
00214     Delegate& operator=(const Delegate& other) noexcept;
00215
00216     constexpr bool operator==(const Delegate& other) const noexcept;
00217     constexpr bool operator!=(const Delegate& other) const noexcept;
00218
00219     template <TRetVal (*TFunction)(TParams...)>
00220     void Bind() noexcept;
00221
00222     template <class TFunction,
00223             typename = typename ::std::enable_if<
00224                 ::std::is_class<TFunction>::value
00225             >::type
00226     >
00227     void Bind(TFunction& functor) noexcept;
00228
00229     template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00230     void Bind(TClass& object) noexcept;
00231
00232     template <TRetVal (*TFunction)(TParams...)>
00233     bool IsBindedTo() const noexcept;
00234
00235     template <class TFunction,
00236             typename = typename ::std::enable_if<
00237                 ::std::is_class<TFunction>::value
00238             >::type

```

```

00275     >
00276     bool IsBindedTo(TFunction& functor) const noexcept;
00277
00279     template <class TClass, TRetVal (TClass::*TMethod) (TParams...)>
00280     bool IsBindedTo(TClass& object) const noexcept;
00281
00283
00284
00289     template <typename... UParams>
00290     TRetVal operator() (UParams&&... params);
00291
00292
00296
00297     template <TRetVal (*TFunction) (TParams...)>
00298     static Delegate CreateAndBind() noexcept;
00299
00301     template <class TFunction,
00302               typename = typename ::std::enable_if<
00303                   ::std::is_class<TFunction>::value
00304               >::type
00305     >
00306     static Delegate CreateAndBind(TFunction& functor) noexcept;
00307
00309     template <class TClass, TRetVal (TClass::*TMethod) (TParams...)>
00310     static Delegate CreateAndBind(TClass& object) noexcept;
00311
00313 private:
00314     using typename DelegateBase<TRetVal, TParams...>::ErasedObjectType;
00315     using typename DelegateBase<TRetVal, TParams...>::StubFnPtr;
00316
00317
00320
00321     template <TRetVal (*TFunction) (TParams...)>
00322     static TRetVal FunctionStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00323
00324     template <class TFunction>
00325     static TRetVal FunctorStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00326
00327     template <class TClass, TRetVal (TClass::*TMethod) (TParams...)>
00328     static TRetVal MethodStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00329
00331 };
00332
00333
00334
00335
00336
00338 //*****
00339 template <typename TRetVal, typename... TParams>
00340 class Delegate<TRetVal (TParams...) const> final: private DelegateBase<TRetVal, TParams...> {
00341 public:
00342     using DelegateBase<TRetVal, TParams...>::IsBinded;
00343     using DelegateBase<TRetVal, TParams...>::operator bool;
00344
00345     using DelegateBase<TRetVal, TParams...>::Unbind;
00346     using DelegateBase<TRetVal, TParams...>::Reset;
00347
00350     constexpr Delegate() noexcept;
00351     constexpr Delegate(const Delegate& other) noexcept;
00353
00354     ~Delegate() noexcept = default;
00355
00356     Delegate& operator=(const Delegate& other) noexcept;
00357
00358
00361     constexpr bool operator==(const Delegate& other) const noexcept;
00362     constexpr bool operator!=(const Delegate& other) const noexcept;
00364
00365
00369
00370     template <TRetVal (*TFunction) (TParams...)>
00371     void Bind() noexcept;
00372
00374     template <class TFunction,
00375               typename = typename ::std::enable_if<
00376                   ::std::is_class<TFunction>::value
00377               >::type
00378     >
00379     void Bind(const TFunction& functor) noexcept;
00380
00381     template <class TClass, TRetVal (TClass::*TMethod) (TParams...) const>
00382     void Bind(const TClass& object) noexcept;
00383
00385
00386
00390

```

```

00391     template <TRetVal (*TFunction)(TParams...)>
00392     bool IsBindedTo() const noexcept;
00393
00394     template <class TFunction,
00395             typename = typename ::std::enable_if<
00396                 ::std::is_class<TFunction>::value
00397             >::type
00398         >
00399     bool IsBindedTo(const TFunction& functor) const noexcept;
00400
00401     template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
00402     bool IsBindedTo(const TClass& object) const noexcept;
00403
00404     template <typename... UParams>
00405     TRetVal operator()(UParams&&... params) const;
00406
00407
00408     template <TRetVal (*TFunction)(TParams...)>
00409     static Delegate CreateAndBind() noexcept;
00410
00411     template <class TFunction,
00412             typename = typename ::std::enable_if<
00413                 ::std::is_class<TFunction>::value
00414             >::type
00415         >
00416     static Delegate CreateAndBind(const TFunction& functor) noexcept;
00417
00418     template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
00419     static Delegate CreateAndBind(const TClass& object) noexcept;
00420
00421 private:
00422     using typename DelegateBase<TRetVal, TParams...>::ErasedObjectType;
00423     using typename DelegateBase<TRetVal, TParams...>::StubFnPtr;
00424
00425     template <TRetVal (*TFunction)(TParams...)>
00426     static TRetVal FunctionStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00427
00428     template <class TFunction>
00429     static TRetVal FunctorStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00430
00431     template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
00432     static TRetVal MethodStub(ErasedObjectType* pTypeErasedObject, TParams... params);
00433 };
00434
00435 //*****
00436 // Inline functions / template implementation
00437
00438 inline const char* BadDelegateCall::what() const noexcept {
00439     return "Bad delegate call";
00440 }
00441
00442 template <typename TRetVal, typename... TParams>
00443 inline constexpr bool DelegateBase<TRetVal, TParams...>::IsBinded() const noexcept {
00444     return pStub_ != &NullStub;
00445 }
00446
00447 template <typename TRetVal, typename... TParams>
00448 inline constexpr DelegateBase<TRetVal, TParams...>::operator bool() const noexcept {
00449     return IsBinded();
00450 }
00451
00452 template <typename TRetVal, typename... TParams>
00453 inline void DelegateBase<TRetVal, TParams...>::Unbind() noexcept {
00454     pStub_ = &NullStub;
00455     pTypeErasedObject_ = nullptr;
00456 }
00457
00458 template <typename TRetVal, typename... TParams>
00459 inline void DelegateBase<TRetVal, TParams...>::Reset() noexcept {
00460     Unbind();
00461 }

```

```

00494
00495
00496 template <typename TRetVal, typename... TParams>
00497 inline constexpr DelegateBase<TRetVal, TParams...>::DelegateBase() noexcept = default;
00498
00499 template <typename TRetVal, typename... TParams>
00500 inline constexpr DelegateBase<TRetVal, TParams...>::DelegateBase(const DelegateBase& other) noexcept =
    default;
00501
00502
00503 template <typename TRetVal, typename... TParams>
00504 inline DelegateBase<TRetVal, TParams...>& DelegateBase<TRetVal, TParams...>::operator=(
00505     const DelegateBase& other) noexcept = default;
00506
00507
00508 template <typename TRetVal, typename... TParams>
00509 inline constexpr bool DelegateBase<TRetVal, TParams...>::operator==(const DelegateBase& other) const
    noexcept {
00510     return (pStub_ == other.pStub_) && (pTypeErasedObject_ == other.pTypeErasedObject_);
00511 }
00512
00513 template <typename TRetVal, typename... TParams>
00514 inline constexpr bool DelegateBase<TRetVal, TParams...>::operator!=(const DelegateBase& other) const
    noexcept {
00515     return !(*this == other);
00516 }
00517
00518
00519 template <typename TRetVal, typename... TParams>
00520 TRetVal DelegateBase<TRetVal, TParams...>::NullStub(ErasedObjectType* pTypeErasedObject, TParams...
    params) {
00521     (void)pTypeErasedObject;
00522     (void)sizeof...(params);
00523
00524     throw BadDelegateCall();
00525 }
00526
00527
00528
00529
00530 template <typename TRetVal, typename... TParams>
00531 inline constexpr Delegate<TRetVal (TParams...)>::Delegate() noexcept = default;
00532
00533 template <typename TRetVal, typename... TParams>
00534 inline constexpr Delegate<TRetVal (TParams...)>::Delegate(const Delegate& other) noexcept = default;
00535
00536
00537 template <typename TRetVal, typename... TParams>
00538 inline Delegate<TRetVal (TParams...)>& Delegate<TRetVal (TParams...)>::operator=(const Delegate&
    other) noexcept = default;
00539
00540
00541 template <typename TRetVal, typename... TParams>
00542 inline constexpr bool Delegate<TRetVal (TParams...)>::operator==(const Delegate& other) const noexcept
    {
00543     return DelegateBase<TRetVal, TParams...>::operator==(other);
00544 }
00545
00546 template <typename TRetVal, typename... TParams>
00547 inline constexpr bool Delegate<TRetVal (TParams...)>::operator!=(const Delegate& other) const noexcept
    {
00548     return !(*this == other);
00549 }
00550
00551
00552 template <typename TRetVal, typename... TParams>
00553 template <TRetVal (*TFunction) (TParams...)>
00554 inline void Delegate<TRetVal (TParams...)>::Bind() noexcept {
00555     this->pStub_ = &FunctionStub<TFunction>;
00556     this->pTypeErasedObject_ = nullptr;
00557 }
00558
00559 template <typename TRetVal, typename... TParams>
00560 template <class TFuncor, typename>
00561 inline void Delegate<TRetVal (TParams...)>::Bind(TFuncor& functor) noexcept {
00562     this->pStub_ = &FuncorStub<TFuncor>;
00563     this->pTypeErasedObject_ = ::std::addressof(functor);
00564 }
00565
00566 template <typename TRetVal, typename... TParams>
00567 template <class TClass, TRetVal (TClass::*TMethod) (TParams...)>
00568 inline void Delegate<TRetVal (TParams...)>::Bind(TClass& object) noexcept {
00569     static_assert(sizeof(TClass*) == sizeof(ErasedObjectType*), "pointers size mismatch");
00570
00571     this->pStub_ = &MethodStub<TClass, TMethod>;
00572     this->pTypeErasedObject_ = ::std::addressof(object);
00573 }

```

```

00574
00575
00576 template <typename TRetVal, typename... TParams>
00577 template <TRetVal (*TFunction)(TParams...)>
00578 inline bool Delegate<TRetVal (TParams...)>::IsBindedTo() const noexcept {
00579     Delegate delegate;
00580     delegate.Bind<TFunction>();
00581     return *this == delegate;
00582 }
00583
00584 template <typename TRetVal, typename... TParams>
00585 template <class TFuncutor, typename>
00586 inline bool Delegate<TRetVal (TParams...)>::IsBindedTo(TFuncutor& functor) const noexcept {
00587     Delegate delegate;
00588     delegate.Bind(functor);
00589     return *this == delegate;
00590 }
00591
00592 template <typename TRetVal, typename... TParams>
00593 template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00594 inline bool Delegate<TRetVal (TParams...)>::IsBindedTo(TClass& object) const noexcept {
00595     Delegate delegate;
00596     delegate.Bind<TClass, TMethod>(object);
00597     return *this == delegate;
00598 }
00599
00600
00601 template <typename TRetVal, typename... TParams>
00602 template <typename... UParams>
00603 inline TRetVal Delegate<TRetVal (TParams...)>::operator()(UParams&&... params) {
00604     return (*this->pStub_)(this->pTypeErasedObject_, ::std::forward<UParams>(params)...);
00605 }
00606
00607
00608 template <typename TRetVal, typename... TParams>
00609 template <TRetVal (*TFunction)(TParams...)>
00610 inline Delegate<TRetVal (TParams...)> Delegate<TRetVal (TParams...)>::CreateAndBind() noexcept {
00611     Delegate delegate;
00612     delegate.Bind<TFunction>();
00613     return delegate;
00614 }
00615
00616 template <typename TRetVal, typename... TParams>
00617 template <class TFuncutor, typename>
00618 inline Delegate<TRetVal (TParams...)> Delegate<TRetVal (TParams...)>::CreateAndBind(TFuncutor& functor)
00619     noexcept {
00619     Delegate delegate;
00620     delegate.Bind(functor);
00621     return delegate;
00622 }
00623
00624 template <typename TRetVal, typename... TParams>
00625 template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00626 inline Delegate<TRetVal (TParams...)> Delegate<TRetVal (TParams...)>::CreateAndBind(TClass& object)
00627     noexcept {
00627     Delegate delegate;
00628     delegate.Bind<TClass, TMethod>(object);
00629     return delegate;
00630 }
00631
00632
00633 template <typename TRetVal, typename... TParams>
00634 template <TRetVal (*TFunction)(TParams...)>
00635 TRetVal Delegate<TRetVal (TParams...)>::FunctionStub(ErasedObjectType* pTypeErasedObject, TParams...
00636     params) {
00636     (void)pTypeErasedObject;
00637
00638     return (*TFunction)(::std::forward<TParams>(params)...);
00639 }
00640
00641 template <typename TRetVal, typename... TParams>
00642 template <class TFuncutor>
00643 TRetVal Delegate<TRetVal (TParams...)>::FuncutorStub(ErasedObjectType* pTypeErasedObject, TParams...
00644     params) {
00644     return (*static_cast<TFuncutor*>(pTypeErasedObject))(::std::forward<TParams>(params)...);
00645 }
00646
00647 template <typename TRetVal, typename... TParams>
00648 template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
00649 TRetVal Delegate<TRetVal (TParams...)>::MethodStub(ErasedObjectType* pTypeErasedObject, TParams...
00650     params) {
00650     return (static_cast<TClass*>(pTypeErasedObject)->TMethod)(::std::forward<TParams>(params)...);
00651 }
00652
00653
00654
00655

```

```

00656 template <typename TRetVal, typename... TParams>
00657 inline constexpr Delegate<TRetVal (TParams...) const>::Delegate() noexcept = default;
00658
00659 template <typename TRetVal, typename... TParams>
00660 inline constexpr Delegate<TRetVal (TParams...) const>::Delegate(const Delegate& other) noexcept =
    default;
00661
00662
00663 template <typename TRetVal, typename... TParams>
00664 inline Delegate<TRetVal (TParams...) const>& Delegate<TRetVal (TParams...) const>::operator=(const
    Delegate& other)
00665     noexcept = default;
00666
00667
00668 template <typename TRetVal, typename... TParams>
00669 inline constexpr bool Delegate<TRetVal (TParams...) const>::operator==(const Delegate& other) const
    noexcept {
00670     return DelegateBase<TRetVal, TParams...>::operator==(other);
00671 }
00672
00673 template <typename TRetVal, typename... TParams>
00674 inline constexpr bool Delegate<TRetVal (TParams...) const>::operator!=(const Delegate& other) const
    noexcept {
00675     return !(*this == other);
00676 }
00677
00678
00679 template <typename TRetVal, typename... TParams>
00680 template <TRetVal (*TFunction) (TParams...)>
00681 inline void Delegate<TRetVal (TParams...) const>::Bind() noexcept {
00682     this->pStub_ = &FunctionStub<TFunction>;
00683     this->pTypeErasedObject_ = nullptr;
00684 }
00685
00686 template <typename TRetVal, typename... TParams>
00687 template <class TFuncor, typename>
00688 inline void Delegate<TRetVal (TParams...) const>::Bind(const TFuncor& functor) noexcept {
00689     this->pStub_ = &FuncorStub<TFuncor>;
00690
00691     // removed const will be restored in FuncorStub()
00692     this->pTypeErasedObject_ = const_cast<ErasedObjectType*>(static_cast<const
    ErasedObjectType*> (::std::addressof(functor)));
00693 }
00694
00695 template <typename TRetVal, typename... TParams>
00696 template <class TClass, TRetVal (TClass::*TMethod) (TParams...) const>
00697 inline void Delegate<TRetVal (TParams...) const>::Bind(const TClass& object) noexcept {
00698     static_assert(sizeof(TClass*) == sizeof(ErasedObjectType*), "pointers size mismatch");
00699
00700     this->pStub_ = &MethodStub<TClass, TMethod>;
00701
00702     // removed const will be restored in MethodStub()
00703     this->pTypeErasedObject_ = const_cast<ErasedObjectType*>(static_cast<const
    ErasedObjectType*> (::std::addressof(object)));
00704 }
00705
00706
00707 template <typename TRetVal, typename... TParams>
00708 template <TRetVal (*TFunction) (TParams...)>
00709 inline bool Delegate<TRetVal (TParams...) const>::IsBindedTo() const noexcept {
00710     Delegate delegate;
00711     delegate.Bind<TFunction>();
00712     return *this == delegate;
00713 }
00714
00715 template <typename TRetVal, typename... TParams>
00716 template <class TFuncor, typename>
00717 inline bool Delegate<TRetVal (TParams...) const>::IsBindedTo(const TFuncor& functor) const noexcept {
00718     Delegate delegate;
00719     delegate.Bind(functor);
00720     return *this == delegate;
00721 }
00722
00723 template <typename TRetVal, typename... TParams>
00724 template <class TClass, TRetVal (TClass::*TMethod) (TParams...) const>
00725 inline bool Delegate<TRetVal (TParams...) const>::IsBindedTo(const TClass& object) const noexcept {
00726     Delegate delegate;
00727     delegate.Bind<TClass, TMethod>(object);
00728     return *this == delegate;
00729 }
00730
00731
00732 template <typename TRetVal, typename... TParams>
00733 template <typename... UParams>
00734 inline TRetVal Delegate<TRetVal (TParams...) const>::operator()(UParams&&... params) const {
00735     return (*this->pStub_)(this->pTypeErasedObject_, ::std::forward<UParams>(params)...);
00736 }

```



```

00737
00738
00739 template <typename TRetVal, typename... TParams>
00740 template <TRetVal (*TFunction)(TParams...)>
00741 inline Delegate<TRetVal (TParams...) const> Delegate<TRetVal (TParams...) const>::CreateAndBind()
    noexcept {
00742     Delegate delegate;
00743     delegate.Bind<TFunction>();
00744     return delegate;
00745 }
00746
00747 template <typename TRetVal, typename... TParams>
00748 template <class TFunction, typename>
00749 inline Delegate<TRetVal (TParams...) const> Delegate<TRetVal (TParams...) const>::CreateAndBind(const
    TFunction& functor)
00750     noexcept {
00751     Delegate delegate;
00752     delegate.Bind(functor);
00753     return delegate;
00754 }
00755
00756 template <typename TRetVal, typename... TParams>
00757 template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
00758 inline Delegate<TRetVal (TParams...) const> Delegate<TRetVal (TParams...) const>::CreateAndBind(const
    TClass& object)
00759     noexcept {
00760     Delegate delegate;
00761     delegate.Bind<TClass, TMethod>(object);
00762     return delegate;
00763 }
00764
00765
00766 template <typename TRetVal, typename... TParams>
00767 template <TRetVal (*TFunction)(TParams...)>
00768 TRetVal Delegate<TRetVal (TParams...) const>::FunctionStub(ErasedObjectType* pTypeErasedObject,
    TParams... params) {
00769     (void)pTypeErasedObject;
00770
00771     return (*TFunction) (::std::forward<TParams>(params)...);
00772 }
00773
00774 template <typename TRetVal, typename... TParams>
00775 template <class TFunction>
00776 TRetVal Delegate<TRetVal (TParams...) const>::FunctorStub(ErasedObjectType* pTypeErasedObject,
    TParams... params) {
00777     // restoring removed const
00778     return (*static_cast<const TFunction*>(pTypeErasedObject)) (::std::forward<TParams>(params)...);
00779 }
00780
00781 template <typename TRetVal, typename... TParams>
00782 template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
00783 TRetVal Delegate<TRetVal (TParams...) const>::MethodStub(ErasedObjectType* pTypeErasedObject,
    TParams... params) {
00784     // restoring removed const
00785     return (static_cast<const
    TClass*>(pTypeErasedObject)->*TMethod) (::std::forward<TParams>(params)...);
00786 }
00787
00788
00789
00790
00791
00792 } // namespace bricxx
00793
00794
00795 #ifdef _MSC_VER
00796 #pragma warning(pop)
00797 #endif
00798
00799
00800 #endif // BRICXX_DELEGATE_HPP_INCLUDED_

```


Index

~Delegate
 bricxx::Delegate< TRetVal(TParams...) const >, 15
 bricxx::Delegate< TRetVal(TParams...) >, 20
~DelegateBase
 bricxx::DelegateBase< TRetVal, TParams >, 24

Bind
 bricxx::Delegate< TRetVal(TParams...) const >, 16
 bricxx::Delegate< TRetVal(TParams...) >, 20, 21

bricxx, 9

bricxx::BadDelegateCall, 11
 what, 11

bricxx::Delegate< TRetVal(TParams...) const >, 13
 ~Delegate, 15
 Bind, 16
 CreateAndBind, 17, 18
 Delegate, 15
 IsBindedTo, 16, 17
 operator!=, 16
 operator(), 17
 operator=, 16
 operator==, 16

bricxx::Delegate< TRetVal(TParams...) >, 18
 ~Delegate, 20
 Bind, 20, 21
 CreateAndBind, 22
 Delegate, 20
 IsBindedTo, 21
 operator!=, 20
 operator(), 21
 operator=, 20
 operator==, 20

bricxx::Delegate< TSignature >, 12

bricxx::DelegateBase< TRetVal, TParams >, 23
 ~DelegateBase, 24
 DelegateBase, 24
 ErasedObjectType, 24
 IsBinded, 25
 NullStub, 25
 operator bool, 25
 operator!=, 25
 operator=, 25
 operator==, 25
 pStub_, 26
 pTypeErasedObject_, 26
 Reset, 25
 StubFnPtr, 24
 Unbind, 25

BRICXX_DELEGATE_API
 delegate.hpp, 27

CreateAndBind
 bricxx::Delegate< TRetVal(TParams...) const >, 17, 18
 bricxx::Delegate< TRetVal(TParams...) >, 22

Delegate
 bricxx::Delegate< TRetVal(TParams...) const >, 15
 bricxx::Delegate< TRetVal(TParams...) >, 20
 delegate.hpp, 27
 BRICXX_DELEGATE_API, 27

DelegateBase
 bricxx::DelegateBase< TRetVal, TParams >, 24

ErasedObjectType
 bricxx::DelegateBase< TRetVal, TParams >, 24

IsBinded
 bricxx::DelegateBase< TRetVal, TParams >, 25

IsBindedTo
 bricxx::Delegate< TRetVal(TParams...) const >, 16, 17
 bricxx::Delegate< TRetVal(TParams...) >, 21

NullStub
 bricxx::DelegateBase< TRetVal, TParams >, 25

operator bool
 bricxx::DelegateBase< TRetVal, TParams >, 25

operator!=
 bricxx::Delegate< TRetVal(TParams...) const >, 16
 bricxx::Delegate< TRetVal(TParams...) >, 20
 bricxx::DelegateBase< TRetVal, TParams >, 25

operator()
 bricxx::Delegate< TRetVal(TParams...) const >, 17
 bricxx::Delegate< TRetVal(TParams...) >, 21

operator=
 bricxx::Delegate< TRetVal(TParams...) const >, 16
 bricxx::Delegate< TRetVal(TParams...) >, 20
 bricxx::DelegateBase< TRetVal, TParams >, 25

operator==
 bricxx::Delegate< TRetVal(TParams...) const >, 16
 bricxx::Delegate< TRetVal(TParams...) >, 20
 bricxx::DelegateBase< TRetVal, TParams >, 25

pStub_
 bricxx::DelegateBase< TRetVal, TParams >, [26](#)
pTypeErasedObject_
 bricxx::DelegateBase< TRetVal, TParams >, [26](#)

Reset
 bricxx::DelegateBase< TRetVal, TParams >, [25](#)

StubFnPtr
 bricxx::DelegateBase< TRetVal, TParams >, [24](#)

Unbind
 bricxx::DelegateBase< TRetVal, TParams >, [25](#)

what
 bricxx::BadDelegateCall, [11](#)